

## Exception Handling and GPIO

1. Reference to the Interrupt Vector Tabel in Appendix A. Declare and code an ISR in C to handle an interrupt request from Port4 pin1. In the ISR, initialize a global variable x of integer data type to 0xFF. Which mode is the processor in when executing the ISR? Assuming that the interrupt was triggered when the processor is executing some user program code, what is the content of the LR register when processor exits the ISR?
2. The user executes the code in Q1 in Code Composer Studio IDE, without touching the hardware, he halted the execution after 5 seconds. The PC halts somewhere in Default\_Handler() routine (see Appendix A). What could be the problem? Or is there a problem?
3. The Port 4 Pin1 interrupt request is triggered when the Bump Switch 1 in the system is pressed. When the user perform the same operation as in Q2, the PC stops somewhere in the infinite while loop in the main() – an expected behaviour. But when the user pressed the Bump switch 1 once before halting the processor, the PC is found to stop in the Default\_Handler(). What could be the problem? Or is there a problem?
4. The stacking operation done before entrance to exception handler only save r0-r3 and r12 registers. What should the user software do to handle the context saving/restore of other data registers?

Q2 Yes, there was a problem with system or other ports. This is because all other ports lead to default handler while port 4 remains in Q1, if the program ends up in default handler, it's a problem outside of Port 4.

Q3 As mentioned in prev question, the system might have a problem, there could be a wrong connection or ISR table is wrong. NOTE THAT ALL ISR entries need to be in the table, regardless of whether the interrupt is utilized or not.

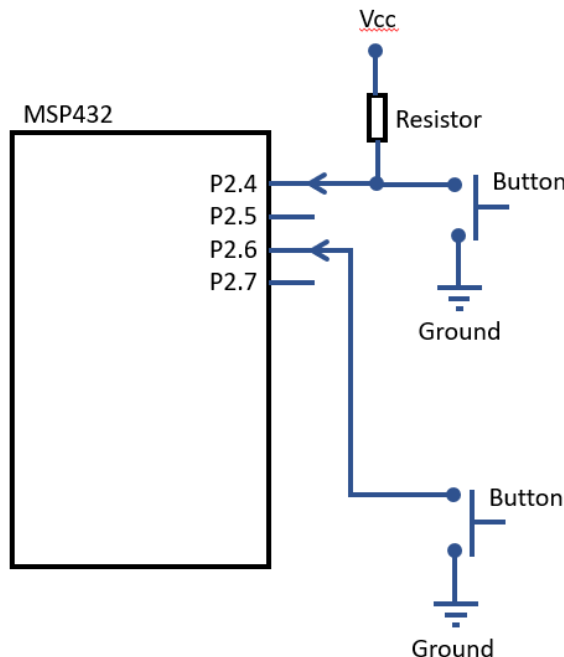
Q4 Proprietary, AAPCS (ARM Arch procedure call standard) understand how various data registers are used between main and main sub-routines. Exception handler is a subroutine. In AAPCS Callee routine is required to save r4-8, r10, r11. r9 is platform specific but in general, there is a requirement the content of r9 has to be persistent across routines. If ISRs are coded in C and compiled with a compiler that is AAPCS compliant, the save/restore of the registers will be done automatically by the compiler.

**Q5**

- GPIO
  - o Port 2.4 and P2.6 (input, internal pullup, falling edge triggered interrupt)
  - o GPIO Input
    - P2SEL0 &= ~(0x50), P2SEL1 &= ~(0x50), P2DIR &= ~(0x50)
  - o Internal Pullup only needed for P2.6. P2.4 has external pullup.
    - P2REN |= (0x40), P2OUT |= 0x40
  - o Interrupt Enable, Clear Interrupt Pending Flag, falling edge triggered.
    - P2IE |= (0x50), P2IFG &= ~(0x50), P2IES |= 0x50
- NVIC
  - o Port 2 interrupt. Location [36] in NVIC table, i.e. bit 4 of ISER1.
  - o NVIC Interrupt enable
    - ISER1 |= (0x10)
  - o Clear any pending bit
    - ICPR1 |= (0x10)
  - o Set priority to 2 (only upper 3 bits are valid for MSP432)
    - IPR3 = (IPR3 & 0xFFFFF00) | 0x00000040

What about PRIMASK and BASEPRI??

5. The diagram below shows the schematic diagram of a MSP432 connecting to two user buttons on P2.4 and P2.6. The system is designed such that an interrupt will be triggered when the button is asserted.



Perform the necessary registers configuration for the GPIO and Timer peripherals

- GPIO
  - P2.4 and 2.6 port pins initialization
  - Interrupt configuration for Port 2.
  - Set priority of the interrupt to 2.
  - Only need to initialize the following registers
    - For GPIO: PxOUT, PxDIR, PxREN, PxSEL0, PxSEL1, PxIES, PxIE and PxIFG.
    - For interrupt: ISER1, ICPR1, IPR9. Why did we choose these registers for Port2?

The necessary documents have been extracted for you and can be found in the appendix of the tutorial.

Q5 change the control bit, look in datasheet change the controlbit manually to initialise the pin. Set it to input pin. P2.6 needs a internal pull up resistor while P2.4 already has a external pull up resistor. PxIE (port x Interrupt Enabler) configure falling edge triggered. PxFG (flag) the flags tells processor if there is an interrupt being queued. ISER register (enabling IRQ, eg ISER1 bit4 leads to  $32+4=36$ , INTISR -> I/O port 2 (can see in data sheet)) IPR9 register (priority selection) calculate based on IRQ look at INTISR[x] then see which byte it corresponds to in IPR9 register. ESSENTIALLY TO CONFIGURE A MICROPROCESSOR, NEED TO READ DATA SHEET AND SET BITS IN DIFFERENT REGISTERS.

## Appendix A

### Interrupt Vector Table and related code sample

```

/*****
 *
 *   MSP432P401R Interrupt Vector Table
 *
 *****/

/* Linker variable that marks the top of the stack. */
extern unsigned long __STACK_END;

/* External declaration for the reset handler that is to be called when the */
/* processor is started */
extern void _c_int00(void);

/* External declaration for system initialization function */
extern void SystemInit(void);

/* Forward declaration of the default fault handlers. */
void Default_Handler      (void) __attribute__((weak));
extern void Reset_Handler (void) __attribute__((weak));

/* Cortex-M4 Processor Exceptions */
extern void NMI_Handler    (void) __attribute__((weak,
alias("Default_Handler")));
extern void HardFault_Handler (void) __attribute__((weak,
alias("Default_Handler")));
extern void MemManage_Handler (void) __attribute__((weak,
alias("Default_Handler")));
extern void BusFault_Handler  (void) __attribute__((weak,
alias("Default_Handler")));
extern void UsageFault_Handler (void) __attribute__((weak,
alias("Default_Handler")));
extern void SVC_Handler       (void) __attribute__((weak,
alias("Default_Handler")));
extern void DebugMon_Handler  (void) __attribute__((weak,
alias("Default_Handler")));
extern void PendSV_Handler    (void) __attribute__((weak,
alias("Default_Handler")));

/* device specific interrupt handler */
extern void SysTick_Handler   (void)
__attribute__((weak, alias("Default_Handler")));
extern void PSS_IRQHandler     (void)
__attribute__((weak, alias("Default_Handler")));
extern void CS_IRQHandler      (void)
__attribute__((weak, alias("Default_Handler")));
extern void PCM_IRQHandler     (void)
__attribute__((weak, alias("Default_Handler")));
extern void WDT_A_IRQHandler   (void)
__attribute__((weak, alias("Default_Handler")));
extern void FPU_IRQHandler     (void)
__attribute__((weak, alias("Default_Handler")));
extern void FLCTL_IRQHandler   (void)
__attribute__((weak, alias("Default_Handler")));
extern void COMP_E0_IRQHandler (void)
__attribute__((weak, alias("Default_Handler")));

```

```

extern void COMP_E1_IRQHandler (void)
__attribute__((weak,alias("Default_Handler")));
extern void TA0_0_IRQHandler (void)
__attribute__((weak,alias("Default_Handler")));
extern void TA0_N_IRQHandler (void)
__attribute__((weak,alias("Default_Handler")));
extern void TA1_0_IRQHandler (void)
__attribute__((weak,alias("Default_Handler")));
extern void TA1_N_IRQHandler (void)
__attribute__((weak,alias("Default_Handler")));
extern void TA2_0_IRQHandler (void)
__attribute__((weak,alias("Default_Handler")));
extern void TA2_N_IRQHandler (void)
__attribute__((weak,alias("Default_Handler")));
extern void TA3_0_IRQHandler (void)
__attribute__((weak,alias("Default_Handler")));
extern void TA3_N_IRQHandler (void)
__attribute__((weak,alias("Default_Handler")));
extern void EUSCIA0_IRQHandler (void)
__attribute__((weak,alias("Default_Handler")));
extern void EUSCIA1_IRQHandler (void)
__attribute__((weak,alias("Default_Handler")));
extern void EUSCIA2_IRQHandler (void)
__attribute__((weak,alias("Default_Handler")));
extern void EUSCIA3_IRQHandler (void)
__attribute__((weak,alias("Default_Handler")));
extern void EUSCIB0_IRQHandler (void)
__attribute__((weak,alias("Default_Handler")));
extern void EUSCIB1_IRQHandler (void)
__attribute__((weak,alias("Default_Handler")));
extern void EUSCIB2_IRQHandler (void)
__attribute__((weak,alias("Default_Handler")));
extern void EUSCIB3_IRQHandler (void)
__attribute__((weak,alias("Default_Handler")));
extern void ADC14_IRQHandler (void)
__attribute__((weak,alias("Default_Handler")));
extern void T32_INT1_IRQHandler (void)
__attribute__((weak,alias("Default_Handler")));
extern void T32_INT2_IRQHandler (void)
__attribute__((weak,alias("Default_Handler")));
extern void T32_INTC_IRQHandler (void)
__attribute__((weak,alias("Default_Handler")));
extern void AES256_IRQHandler (void)
__attribute__((weak,alias("Default_Handler")));
extern void RTC_C_IRQHandler (void)
__attribute__((weak,alias("Default_Handler")));
extern void DMA_ERR_IRQHandler (void)
__attribute__((weak,alias("Default_Handler")));
extern void DMA_INT3_IRQHandler (void)
__attribute__((weak,alias("Default_Handler")));
extern void DMA_INT2_IRQHandler (void)
__attribute__((weak,alias("Default_Handler")));
extern void DMA_INT1_IRQHandler (void)
__attribute__((weak,alias("Default_Handler")));
extern void DMA_INT0_IRQHandler (void)
__attribute__((weak,alias("Default_Handler")));
extern void PORT1_IRQHandler (void)
__attribute__((weak,alias("Default_Handler")));

```

```

extern void PORT2_IRQHandler (void)
__attribute__((weak,alias("Default_Handler")));
extern void PORT3_IRQHandler (void)
__attribute__((weak,alias("Default_Handler")));
extern void PORT4_IRQHandler (void)
__attribute__((weak,alias("Default_Handler")));
extern void PORT5_IRQHandler (void)
__attribute__((weak,alias("Default_Handler")));
extern void PORT6_IRQHandler (void)
__attribute__((weak,alias("Default_Handler")));

/* Interrupt vector table. */
#pragma RETAIN(interruptVectors)
#pragma DATA_SECTION(interruptVectors, ".intvecs")
void (* const interruptVectors[])(void) =
{
    (void (*)(void))((uint32_t)&__STACK_END),
    Reset_Handler,          /* The initial stack pointer */
    NMI_Handler,            /* The reset handler */
    HardFault_Handler,      /* The NMI handler */
    MemManage_Handler,      /* The hard fault handler */
    BusFault_Handler,       /* The MPU fault handler */
    UsageFault_Handler,     /* The bus fault handler */
    0,                      /* The usage fault handler */
    0,                      /* Reserved */
    0,                      /* Reserved */
    0,                      /* Reserved */
    0,                      /* Reserved */
    SVC_Handler,           /* SVC call handler */
    DebugMon_Handler,      /* Debug monitor handler */
    0,                      /* Reserved */
    PendSV_Handler,        /* The PendSV handler */
    SysTick_Handler,       /* The SysTick handler */
    PSS_IRQHandler,        /* PSS ISR */
    CS_IRQHandler,         /* CS ISR */
    PCM_IRQHandler,        /* PCM ISR */
    WDT_A_IRQHandler,      /* WDT ISR */
    FPU_IRQHandler,        /* FPU ISR */
    FLCTL_IRQHandler,      /* FLCTL ISR */
    COMP_E0_IRQHandler,    /* COMP0 ISR */
    COMP_E1_IRQHandler,    /* COMP1 ISR */
    TA0_0_IRQHandler,      /* TA0_0 ISR */
    TA0_N_IRQHandler,      /* TA0_N ISR */
    TA1_0_IRQHandler,      /* TA1_0 ISR */
    TA1_N_IRQHandler,      /* TA1_N ISR */
    TA2_0_IRQHandler,      /* TA2_0 ISR */
    TA2_N_IRQHandler,      /* TA2_N ISR */
    TA3_0_IRQHandler,      /* TA3_0 ISR */
    TA3_N_IRQHandler,      /* TA3_N ISR */
    EUSCIA0_IRQHandler,     /* EUSCIA0 ISR */
    EUSCIA1_IRQHandler,     /* EUSCIA1 ISR */
    EUSCIA2_IRQHandler,     /* EUSCIA2 ISR */
    EUSCIA3_IRQHandler,     /* EUSCIA3 ISR */
    EUSCIB0_IRQHandler,     /* EUSCIB0 ISR */
    EUSCIB1_IRQHandler,     /* EUSCIB1 ISR */
    EUSCIB2_IRQHandler,     /* EUSCIB2 ISR */
    EUSCIB3_IRQHandler,     /* EUSCIB3 ISR */
    ADC14_IRQHandler,      /* ADC14 ISR */
    T32_INT1_IRQHandler,    /* T32_INT1 ISR */

```

```

T32_INT2_IRQHandler,      /* T32_INT2 ISR      */
T32_INTC_IRQHandler,      /* T32_INTC ISR      */
AES256_IRQHandler,        /* AES ISR           */
RTC_C_IRQHandler,         /* RTC ISR           */
DMA_ERR_IRQHandler,       /* DMA_ERR ISR       */
DMA_INT3_IRQHandler,      /* DMA_INT3 ISR      */
DMA_INT2_IRQHandler,      /* DMA_INT2 ISR      */
DMA_INT1_IRQHandler,      /* DMA_INT1 ISR      */
DMA_INT0_IRQHandler,      /* DMA_INT0 ISR      */
PORT1_IRQHandler,         /* PORT1 ISR         */
PORT2_IRQHandler,         /* PORT2 ISR         */
PORT3_IRQHandler,         /* PORT3 ISR         */
PORT4_IRQHandler,         /* PORT4 ISR         */
PORT5_IRQHandler,         /* PORT5 ISR         */
PORT6_IRQHandler,         /* PORT6 ISR         */
};

                                                                    */

void Reset_Handler(void)
{
    SystemInit();

    /* Jump to the CCS C Initialization Routine. */
    __asm("    .global _c_int00\n"
          "    b.w    _c_int00");
}

                                                                    */

void Default_Handler(void)
{
    /* Enter an infinite loop. */
    while(1)
    {
    }
}

```

## Appendix B

### GPIO

Table 6-67. Port P2 (P2.4 to P2.7) Pin Functions

| PIN NAME (P2.x)              | x | FUNCTION   | CONTROL BITS OR SIGNALS <sup>(1)</sup> |          |          |         |
|------------------------------|---|------------|----------------------------------------|----------|----------|---------|
|                              |   |            | P2DIR.x                                | P2SEL1.x | P2SEL0.x | P2MAPx  |
| P2.4/PM_TA0.1 <sup>(2)</sup> | 4 | P2.4 (I/O) | I: 0; O: 1                             | 0        | 0        | X       |
|                              |   | TA0.CCI1A  | 0                                      | 0        | 1        | default |
|                              |   | TA0.1      | 1                                      |          |          |         |
|                              |   | N/A        | 0                                      | 1        | 0        | X       |
|                              |   | DVSS       | 1                                      |          |          |         |
|                              |   | N/A        | 0                                      | 1        | 1        | X       |
|                              |   | DVSS       | 1                                      |          |          |         |
| P2.5/PM_TA0.2 <sup>(2)</sup> | 5 | P2.5 (I/O) | I: 0; O: 1                             | 0        | 0        | X       |
|                              |   | TA0.CCI2A  | 0                                      | 0        | 1        | default |
|                              |   | TA0.2      | 1                                      |          |          |         |
|                              |   | N/A        | 0                                      | 1        | 0        | X       |
|                              |   | DVSS       | 1                                      |          |          |         |
|                              |   | N/A        | 0                                      | 1        | 1        | X       |
|                              |   | DVSS       | 1                                      |          |          |         |
| P2.6/PM_TA0.3 <sup>(2)</sup> | 6 | P2.6 (I/O) | I: 0; O: 1                             | 0        | 0        | X       |
|                              |   | TA0.CCI3A  | 0                                      | 0        | 1        | default |
|                              |   | TA0.3      | 1                                      |          |          |         |
|                              |   | N/A        | 0                                      | 1        | 0        | X       |
|                              |   | DVSS       | 1                                      |          |          |         |
|                              |   | N/A        | 0                                      | 1        | 1        | X       |
|                              |   | DVSS       | 1                                      |          |          |         |
| P2.7/PM_TA0.4 <sup>(2)</sup> | 7 | P2.7 (I/O) | I: 0; O: 1                             | 0        | 0        | X       |
|                              |   | TA0.CCI4A  | 0                                      | 0        | 1        | default |
|                              |   | TA0.4      | 1                                      |          |          |         |
|                              |   | N/A        | 0                                      | 1        | 0        | X       |
|                              |   | DVSS       | 1                                      |          |          |         |
|                              |   | N/A        | 0                                      | 1        | 1        | X       |
|                              |   | DVSS       | 1                                      |          |          |         |

(1) X = don't care

(2) Not available on the 64-pin RGC package.

#### 12.4.3 PxOUT Register

Port X Output Register (X = 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 or J)

Figure 12-3. PxOUT Register

| 7     | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
|-------|----|----|----|----|----|----|----|
| PxOUT |    |    |    |    |    |    |    |
| rw    | rw | rw | rw | rw | rw | rw | rw |

Table 12-6. PxOUT Register Description

| Bit | Field | Type | Reset     | Description                                                                                                                                                                                                                |
|-----|-------|------|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7-0 | PxOUT | RW   | Undefined | Port X output.<br>When I/O configured to output mode:<br>0b = Output is low.<br>1b = Output is high.<br>When I/O configured to input mode and pullups/pulldowns enabled:<br>0b = Pulldown selected<br>1b = Pullup selected |

### 12.4.4 PxDIR Register

Port X Direction Register (X = 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 or J)

Figure 12-4. PxDIR Register

| 7     | 6    | 5    | 4    | 3    | 2    | 1    | 0    |
|-------|------|------|------|------|------|------|------|
| PxDIR |      |      |      |      |      |      |      |
| rw-0  | rw-0 | rw-0 | rw-0 | rw-0 | rw-0 | rw-0 | rw-0 |

Table 12-7. PxDIR Register Description

| Bit | Field | Type | Reset | Description                                                                          |
|-----|-------|------|-------|--------------------------------------------------------------------------------------|
| 7-0 | PxDIR | RW   | 0h    | Port X direction.<br>0b = Port configured as input<br>1b = Port configured as output |

### 12.4.5 PxREN Register

Port X Pullup or Pulldown Resistor Enable Register (X = 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 or J)

Figure 12-5. PxREN Register

| 7     | 6    | 5    | 4    | 3    | 2    | 1    | 0    |
|-------|------|------|------|------|------|------|------|
| PxREN |      |      |      |      |      |      |      |
| rw-0  | rw-0 | rw-0 | rw-0 | rw-0 | rw-0 | rw-0 | rw-0 |

Table 12-8. PxREN Register Description

| Bit | Field | Type | Reset | Description                                                                                                                                                                                                             |
|-----|-------|------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7-0 | PxREN | RW   | 0h    | Port X pullup or pulldown resistor enable. When the port is configured as an input, setting this bit enables or disables the pullup or pulldown.<br>0b = Pullup or pulldown disabled<br>1b = Pullup or pulldown enabled |

### 12.4.7 PxSEL0 Register

Port X Function Selection Register 0 (X = 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 or J)

Figure 12-7. PxSEL0 Register

| 7      | 6    | 5    | 4    | 3    | 2    | 1    | 0    |
|--------|------|------|------|------|------|------|------|
| PxSEL0 |      |      |      |      |      |      |      |
| rw-0   | rw-0 | rw-0 | rw-0 | rw-0 | rw-0 | rw-0 | rw-0 |

Table 12-10. PxSEL0 Register Description

| Bit | Field  | Type | Reset | Description                                                                                                                                                                                                                                                                                                                |
|-----|--------|------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7-0 | PxSEL0 | RW   | 0h    | Port function selection. Each bit corresponds to one channel on Port X. The values of each bit position in PxSEL1 and PxSEL0 are combined to specify the function. For example, if P1SEL1.5 = 1 and P1SEL0.5 = 0, then the secondary module function is selected for P1.5.<br>See PxSEL1 for the definition of each value. |



### 12.4.8 PxSEL1 Register

Port X Function Selection Register 1 (X = 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 or J)

**Figure 12-8. PxSEL1 Register**

| 7      | 6    | 5    | 4    | 3    | 2    | 1    | 0    |
|--------|------|------|------|------|------|------|------|
| PxSEL1 |      |      |      |      |      |      |      |
| rw-0   | rw-0 | rw-0 | rw-0 | rw-0 | rw-0 | rw-0 | rw-0 |

**Table 12-11. PxSEL1 Register Description**

| Bit | Field  | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|-----|--------|------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7-0 | PxSEL1 | RW   | 0h    | Port function selection. Each bit corresponds to one channel on Port X. The values of each bit position in PxSEL1 and PxSEL0 are combined to specify the function. For example, if P1SEL1.5 = 1 and P1SEL0.5 = 0, then the secondary module function is selected for P1.5.<br>00b = General-purpose I/O is selected<br>01b = Primary module function is selected<br>10b = Secondary module function is selected<br>11b = Tertiary module function is selected |

### 12.4.10 PxIES Register

Port X Interrupt Edge Select Register (X = 1, 2, 3, 4, 5, 6, 7, 8, 9 or 10)

**Figure 12-10. PxIES Register**

| 7     | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
|-------|----|----|----|----|----|----|----|
| PxIES |    |    |    |    |    |    |    |
| rw    | rw | rw | rw | rw | rw | rw | rw |

**Table 12-13. P1IES Register Description**

| Bit | Field | Type | Reset     | Description                                                                                                                                    |
|-----|-------|------|-----------|------------------------------------------------------------------------------------------------------------------------------------------------|
| 7-0 | PxIES | RW   | Undefined | Port X interrupt edge select<br>0b = PxIFG flag is set with a low-to-high transition.<br>1b = PxIFG flag is set with a high-to-low transition. |

### 12.4.11 PxIE Register

Port X Interrupt Enable Register (X = 1, 2, 3, 4, 5, 6, 7, 8, 9 or 10)

**Figure 12-11. PxIE Register**

| 7    | 6    | 5    | 4    | 3    | 2    | 1    | 0    |
|------|------|------|------|------|------|------|------|
| PxIE |      |      |      |      |      |      |      |
| rw-0 | rw-0 | rw-0 | rw-0 | rw-0 | rw-0 | rw-0 | rw-0 |

**Table 12-14. PxIE Register Description**

| Bit | Field | Type | Reset | Description                                                                                                        |
|-----|-------|------|-------|--------------------------------------------------------------------------------------------------------------------|
| 7-0 | PxIE  | RW   | 0h    | Port X interrupt enable<br>0b = Corresponding port interrupt disabled<br>1b = Corresponding port interrupt enabled |

**12.4.12 PxIFG Register**

Port X Interrupt Flag Register (X = 1, 2, 3, 4, 5, 6, 7, 8, 9 or 10)

**Figure 12-12. PxIFG Register**

| 7     | 6    | 5    | 4    | 3    | 2    | 1    | 0    |
|-------|------|------|------|------|------|------|------|
| PxIFG |      |      |      |      |      |      |      |
| rw-0  | rw-0 | rw-0 | rw-0 | rw-0 | rw-0 | rw-0 | rw-0 |

**Table 12-15. PxIFG Register Description**

| Bit | Field | Type | Reset | Description                                                                          |
|-----|-------|------|-------|--------------------------------------------------------------------------------------|
| 7-0 | PxIFG | RW   | 0h    | Port X interrupt flag<br>0b = No interrupt is pending.<br>1b = Interrupt is pending. |

## NVIC

### NVIC Table (P118 MSP432 Datasheets)

Table 6-39. NVIC Interrupts (continued)

| NVIC INTERRUPT INPUT | SOURCE                  | FLAGS IN SOURCE                                                                                 |
|----------------------|-------------------------|-------------------------------------------------------------------------------------------------|
| INTISR[4]            | FPU_INT <sup>(2)</sup>  | Combined interrupt from flags in the FPSCR (part of Cortex-M4 FPU)                              |
| INTISR[5]            | FLCTL                   | Flash Controller interrupt flags                                                                |
| INTISR[6]            | COMP_E0                 | Comparator_E0 interrupt flags                                                                   |
| INTISR[7]            | COMP_E1                 | Comparator_E1 interrupt flags                                                                   |
| INTISR[8]            | Timer_A0                | TA0CCTL0.CCIFG                                                                                  |
| INTISR[9]            | Timer_A0                | TA0CCTLx.CCIFG (x = 1 to 4), TA0CTL.TAIFG                                                       |
| INTISR[10]           | Timer_A1                | TA1CCTL0.CCIFG                                                                                  |
| INTISR[11]           | Timer_A1                | TA1CCTLx.CCIFG (x = 1 to 4), TA1CTL.TAIFG                                                       |
| INTISR[12]           | Timer_A2                | TA2CCTL0.CCIFG                                                                                  |
| INTISR[13]           | Timer_A2                | TA2CCTLx.CCIFG (x = 1 to 4), TA2CTL.TAIFG                                                       |
| INTISR[14]           | Timer_A3                | TA3CCTL0.CCIFG                                                                                  |
| INTISR[15]           | Timer_A3                | TA3CCTLx.CCIFG (x = 1 to 4), TA3CTL.TAIFG                                                       |
| INTISR[16]           | eUSCI_A0                | UART or SPI mode TX, RX, and Status Flags                                                       |
| INTISR[17]           | eUSCI_A1                | UART or SPI mode TX, RX, and Status Flags                                                       |
| INTISR[18]           | eUSCI_A2                | UART or SPI mode TX, RX, and Status Flags                                                       |
| INTISR[19]           | eUSCI_A3                | UART or SPI mode TX, RX, and Status Flags                                                       |
| INTISR[20]           | eUSCI_B0                | SPI or I <sup>2</sup> C mode TX, RX, and Status Flags (I <sup>2</sup> C in multiple-slave mode) |
| INTISR[21]           | eUSCI_B1                | SPI or I <sup>2</sup> C mode TX, RX, and Status Flags (I <sup>2</sup> C in multiple-slave mode) |
| INTISR[22]           | eUSCI_B2                | SPI or I <sup>2</sup> C mode TX, RX, and Status Flags (I <sup>2</sup> C in multiple-slave mode) |
| INTISR[23]           | eUSCI_B3                | SPI or I <sup>2</sup> C mode TX, RX, and Status Flags (I <sup>2</sup> C in multiple-slave mode) |
| INTISR[24]           | Precision ADC           | IFG[0-31], LO/INH/IFG, RDYIFG, OVIFG, TOVIFG                                                    |
| INTISR[25]           | Timer32_INT1            | Timer32 interrupt for Timer1                                                                    |
| INTISR[26]           | Timer32_INT2            | Timer32 interrupt for Timer2                                                                    |
| INTISR[27]           | Timer32_INT3            | Timer32 Combined Interrupt                                                                      |
| INTISR[28]           | AES256                  | AESRDYIFG                                                                                       |
| INTISR[29]           | RTC_C                   | OFIFG, RDYIFG, TEVIFG, AIFG, RT0PSIFG, RT1PSIFG                                                 |
| INTISR[30]           | DMA_ERR                 | DMA error interrupt                                                                             |
| INTISR[31]           | DMA_INT3                | DMA completion interrupt3                                                                       |
| INTISR[32]           | DMA_INT2                | DMA completion interrupt2                                                                       |
| INTISR[33]           | DMA_INT1                | DMA completion interrupt1                                                                       |
| INTISR[34]           | DMA_INT0 <sup>(3)</sup> | DMA completion interrupt0                                                                       |
| INTISR[35]           | I/O Port P1             | P1IFG.x (x = 0 to 7)                                                                            |
| INTISR[36]           | I/O Port P2             | P2IFG.x (x = 0 to 7)                                                                            |
| INTISR[37]           | I/O Port P3             | P3IFG.x (x = 0 to 7)                                                                            |
| INTISR[38]           | I/O Port P4             | P4IFG.x (x = 0 to 7)                                                                            |
| INTISR[39]           | I/O Port P5             | P5IFG.x (x = 0 to 7)                                                                            |
| INTISR[40]           | I/O Port P6             | P6IFG.x (x = 0 to 7)                                                                            |
| INTISR[41]           | Reserved                |                                                                                                 |
| INTISR[42]           | Reserved                |                                                                                                 |

**2.4.3.2 ISER1 Register (Offset = 104h) [reset = 00000000h]**

ISER1 is shown in [Figure 2-21](#) and described in [Table 2-27](#).

Irq 32 to 63 Set Enable Register. Use the Interrupt Set-Enable Registers to enable interrupts and determine which interrupts are currently enabled.

**Figure 2-21. ISER1 Register**

|        |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |
|--------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|---|---|---|---|---|---|---|---|---|---|
| 31     | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| SETENA |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |
| R/W-0h |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |

**Table 2-27. ISER1 Register Field Descriptions**

| Bit  | Field  | Type | Reset | Description                                                                                                                                                                        |
|------|--------|------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31-0 | SETENA | R/W  | 0h    | Writing 0 to a SETENA bit has no effect, writing 1 to a bit enables the corresponding interrupt. Reading the bit returns its current enable state. Reset clears the SETENA fields. |

**2.4.3.8 ICPR1 Register (Offset = 284h) [reset = 00000000h]**

ICPR1 is shown in [Figure 2-27](#) and described in [Table 2-33](#).

Irq 32 to 63 Clear Pending Register. Use the Interrupt Clear-Pending Registers to clear pending interrupts and determine which interrupts are currently pending.

**Figure 2-27. ICPR1 Register**

|         |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |
|---------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|---|---|---|---|---|---|---|---|---|---|
| 31      | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| CLRPEND |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |
| R/W-0h  |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |

**Table 2-33. ICPR1 Register Field Descriptions**

| Bit  | Field   | Type | Reset | Description                                                                                                                                         |
|------|---------|------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| 31-0 | CLRPEND | R/W  | 0h    | Writing 0 to a CLRPEND bit has no effect, writing 1 to a bit clears the corresponding pending interrupt. Reading the bit returns its current state. |

**2.4.3.20 IPR9 Register (Offset = 424h) [reset = 00000000h]**

IPR9 is shown in [Figure 2-39](#) and described in [Table 2-45](#).

Irq 36 to 39 Priority Register. Use the Interrupt Priority Registers to assign a priority from 0 to 255 to each of the available interrupts. 0 is the highest priority, and 255 is the lowest.

The hardware priority mechanism only looks at the upper N bits of the priority field (where N is 3 for the MSP432 family), so any prioritization must be performed in those bits. The remaining bits can be used to sub-prioritize the interrupt sources, and may be used by the hardware priority mechanism on a future part

**Figure 2-39. IPR9 Register**

|        |    |    |    |    |    |    |    |        |    |    |    |    |    |    |    |        |    |    |    |    |    |   |   |        |   |   |   |   |   |   |   |
|--------|----|----|----|----|----|----|----|--------|----|----|----|----|----|----|----|--------|----|----|----|----|----|---|---|--------|---|---|---|---|---|---|---|
| 31     | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23     | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15     | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7      | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| PRI_39 |    |    |    |    |    |    |    | PRI_38 |    |    |    |    |    |    |    | PRI_37 |    |    |    |    |    |   |   | PRI_36 |   |   |   |   |   |   |   |
| R/W-0h |    |    |    |    |    |    |    | R/W-0h |    |    |    |    |    |    |    | R/W-0h |    |    |    |    |    |   |   | R/W-0h |   |   |   |   |   |   |   |

**Table 2-45. IPR9 Register Field Descriptions**

| Bit   | Field  | Type | Reset | Description              |
|-------|--------|------|-------|--------------------------|
| 31-24 | PRI_39 | R/W  | 0h    | Priority of interrupt 39 |
| 23-16 | PRI_38 | R/W  | 0h    | Priority of interrupt 38 |
| 15-8  | PRI_37 | R/W  | 0h    | Priority of interrupt 37 |
| 7-0   | PRI_36 | R/W  | 0h    | Priority of interrupt 36 |